Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Адаптивный шаг обучения в однослойном линейном персептроне (метод наискорейшего спуска)**»**

Выполнил:
Студент 3 курса
Группы ИИ-26
Хомиченко И.А.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется.

Задачи лабораторной работы:

Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).

Применить вычисленный α(t) для обновления весов ij и порогов Tj согласно дельта-правилу.

Используя данные своего варианта, провести два эксперимента:

- Обучение с фиксированным шагом (например, α=0.1 или α=1p).
- Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки Es≤Ee.

Построить графики обучения Es(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.

Выполнить графическую визуализацию разделяющей линии для адаптивного метода.

4. Реализовать режим функционирования сети:

- пользователь задаёт произвольный входной вектор,
- сеть вычисляет выходной класс,
- соответствующая точка отображается на графике,

Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

Вариант 8

x1, x2 - входные данные сети, e - эталонные значения

x_1	x_2	e
5	6	0
-5	6	1
5	-6	1
-5	-6	1

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

points_data = np.array([
    [5, 6],
    [-5, 6],
    [5, -6],
    [-5, -6]
])

target_data = np.array([0, 1, 1, 1]).reshape(-1, 1)


def model_output(weights, inputs, bias):
    return inputs @ weights - bias
```

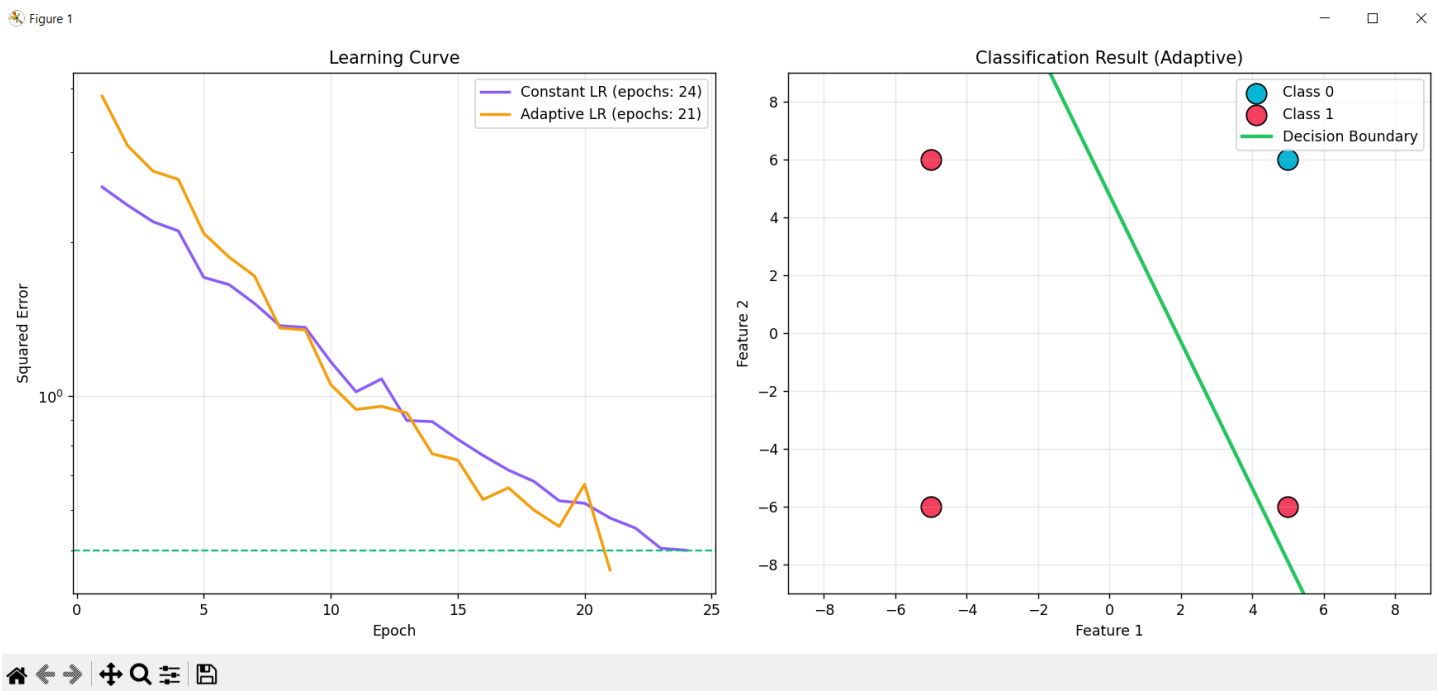
```python
def train_constant(dataset, labels, lr=0.01, threshold=0.5, epochs_limit=20000):
    sample_count, feature_count = dataset.shape
    weights_vec = np.zeros((feature_count, 1))
    bias_term = 0.0
    error_track = []
    final_epoch = epochs_limit

    print(f"[Constant LR] Started | lr={lr} | target error={threshold}")

    for epoch in range(epochs_limit):
        shuffled_idx = np.random.permutation(sample_count)

        for i in shuffled_idx:
            sample = dataset[i:i+1]
            label = float(labels[i, 0])

            raw = model_output(weights_vec, sample, bias_term)
            delta = float(raw[0, 0]) - label

            weights_vec -= lr * sample.T * delta
            bias_term += lr * delta

        predictions = model_output(weights_vec, dataset, bias_term)
        total_error = float(np.sum((predictions - labels) ** 2))
        error_track.append(total_error)

        if epoch % 40 == 0 and epoch > 0:
            print(f"Epoch {epoch:5d} | Error = {total_error:.6f}")

        if total_error <= threshold:
            final_epoch = epoch + 1
            print(f"Training stopped at epoch {final_epoch} | Error={total_error:.6f}\n")
            break
    else:
        print(f"Reached max epochs ({epochs_limit}) | Error={error_track[-1]:.6f}\n")

    return weights_vec, bias_term, error_track, final_epoch


def train_adaptive(dataset, labels, threshold=0.5, epochs_limit=20000):
    sample_count, feature_count = dataset.shape
    weights_vec = np.zeros((feature_count, 1))
    bias_term = 0.0
    error_track = []
    final_epoch = epochs_limit

    print(f"[Adaptive LR] Started | target error={threshold}")

    for epoch in range(epochs_limit):
        shuffled_idx = np.random.permutation(sample_count)

        for i in shuffled_idx:
            sample = dataset[i:i+1]
            label = float(labels[i, 0])

            norm_value = float(np.sum(sample ** 2))
            dynamic_lr = 1.0 / norm_value if norm_value > 1e-12 else 0.0

            raw = model_output(weights_vec, sample, bias_term)
            delta = float(raw[0, 0]) - label

            weights_vec -= dynamic_lr * sample.T * delta
```

```
            bias_term += dynamic_lr * delta

        predictions = model_output(weights_vec, dataset, bias_term)
        total_error = float(np.sum((predictions - labels) ** 2))
        error_track.append(total_error)

        if epoch % 8 == 0 and epoch > 0:
            print(f"Epoch {epoch:5d} | Error = {total_error:.6f}")

        if total_error <= threshold:
            final_epoch = epoch + 1
            print(f"Training stopped at epoch {final_epoch} | Error={total_error:.6f}\n")
            break
    else:
        print(f"Reached max epochs ({epochs_limit}) | Error={error_track[-1]:.6f}\n")

    return weights_vec, bias_term, error_track, final_epoch


np.random.seed(42)
error_target = 0.5

w_const, b_const, err_const, ep_const = train_constant(points_data, target_data, lr=0.01,
threshold=error_target)
np.random.seed(42)
w_adapt, b_adapt, err_adapt, ep_adapt = train_adaptive(points_data, target_data, thresh-
old=error_target)

print(f"Speed ratio: {ep_const} / {ep_adapt} = {ep_const / ep_adapt:.2f}")

figure, (plot_loss, plot_space) = plt.subplots(1, 2, figsize=(14, 6))

plot_loss.plot(np.arange(1, len(err_const) + 1), err_const,
               linewidth=2, color="#8B5CF6",
               label=f"Constant LR (epochs: {ep_const})")

plot_loss.plot(np.arange(1, len(err_adapt) + 1), err_adapt,
               linewidth=2, color="#F59E0B",
               label=f"Adaptive LR (epochs: {ep_adapt})")

plot_loss.axhline(error_target, linestyle="--",
                  linewidth=1.3, color="#10B981")

plot_loss.set_xlabel("Epoch")
plot_loss.set_ylabel("Squared Error")
plot_loss.set_yscale("log")
plot_loss.set_title("Learning Curve")
plot_loss.legend()
plot_loss.grid(alpha=0.3)

mask_zero = target_data[:, 0] == 0
mask_one = target_data[:, 0] == 1

plot_space.scatter(points_data[mask_zero][:, 0],
                   points_data[mask_zero][:, 1],
                   s=200, color="#06B6D4",
                   edgecolors="black", label="Class 0")

plot_space.scatter(points_data[mask_one][:, 0],
                   points_data[mask_one][:, 1],
                   s=200, color="#F43F5E",
                   edgecolors="black", label="Class 1")
```

```python
weight1, weight2 = w_adapt.flatten()
x_line = np.linspace(-9, 9, 500)

if abs(weight2) > 1e-9:
    y_line = (b_adapt + 0.5 - weight1 * x_line) / weight2
    plot_space.plot(x_line, y_line,
                    linewidth=2.5, color="#22C55E",
                    label="Decision Boundary")

plot_space.set_xlim(-9, 9)
plot_space.set_ylim(-9, 9)
plot_space.set_xlabel("Feature 1")
plot_space.set_ylabel("Feature 2")
plot_space.set_title("Classification Result (Adaptive)")
plot_space.legend()
plot_space.grid(alpha=0.3)

plt.tight_layout()
plt.show()

history_points = []
history_classes = []

print("-" * 40)
print("Prediction mode (Adaptive model)")
print("Enter coordinates: x1 x2   or type 'exit'\n")

while True:
    user_entry = input("Input: ").strip()

    if user_entry in ("exit", ""):
        break

    try:
        val1, val2 = map(float, user_entry.split())
        sample = np.array([[val1, val2]])

        result_score = model_output(w_adapt, sample, b_adapt).item()
        predicted_class = 1 if result_score >= 0.5 else 0

        print(f"Score: {result_score:+.6f}")
        print(f"Predicted class: {predicted_class}\n")

        history_points.append([val1, val2])
        history_classes.append(predicted_class)

        fig2, axis2 = plt.subplots(figsize=(8, 8))

        axis2.scatter(points_data[mask_zero][:, 0],
                      points_data[mask_zero][:, 1],
                      s=200, color="#06B6D4",
                      edgecolors="black", label="Class 0")

        axis2.scatter(points_data[mask_one][:, 0],
                      points_data[mask_one][:, 1],
                      s=200, color="#F43F5E",
                      edgecolors="black", label="Class 1")

        if abs(weight2) > 1e-9:
            y_dynamic = (b_adapt + 0.5 - weight1 * x_line) / weight2
            axis2.plot(x_line, y_dynamic,
                       linewidth=2.5, color="#22C55E",
                       label="Boundary")
```

```
        for pt, cls in zip(history_points, history_classes):
            axis2.scatter(pt[0], pt[1],
                          marker="X", s=260,
                          color="#FACC15" if cls == 1 else "#1E3A8A",
                          edgecolors="black", zorder=6)

        axis2.set_xlim(-9, 9)
        axis2.set_ylim(-9, 9)
        axis2.set_xlabel("Feature 1")
        axis2.set_ylabel("Feature 2")
        axis2.set_title(f"Last input: ({val1}, {val2}) → class {predicted_class}")
        axis2.legend()
        axis2.grid(alpha=0.3)

        plt.tight_layout()
        plt.show()

    except Exception:
        print("Invalid input format. Use: number number\n")

print("Program finished.")
```

Результат программы:



```
[Constant LR] Started | lr=0.01 | target error=0.5
Training stopped at epoch 24 | Error=0.499973

[Adaptive LR] Started | target error=0.5
Epoch     8 | Error = 1.348265
Epoch    16 | Error = 0.662407
Training stopped at epoch 21 | Error=0.457671

Speed ratio: 24 / 21 = 1.14
----------------------------------------
Prediction mode (Adaptive model)
Enter coordinates: x1 x2   or type 'exit'
```

Вывод: изучил адаптивный шаг обучения в однослойном линейном персептроне (метод наискорейшего спуска